API Documentation

Property Platform API: find properties by interior preferences and budget (NoBroker Menternship)
Backend: Node.js, Express, Mongoose (MongoDB) | Prepared by: C Sri Harsha

1. Overview

Item	Value
Base URL	http://localhost:3000 (change the port with PORT in .env)
Format	JSON for requests and answers. Header: `Content-Type: application/json`. Image upload uses multipart form data.
Login	Send the token from login or signup in a header: `Authorization: Bearer <token>`. Tokens last 7 days.
Database	MongoDB, database `property_platform` (collections: properties, users, userPreferences, revokedTokens)
Paging	`page` and `limit` (default 10, max 50). Lists answer with `total`, `page`, `limit`, `pages`.
Images	Uploaded images are public files at `/uploads/properties/<name>`
Security	helmet headers, CORS, JSON body limit 100 KB, login rate limit, bcrypt passwords, JWT (HS256) with logout (revoked tokens), safe search values

Long lists such as `rooms` are shortened to `["..."]` in the examples.

Every error has the same shape: `{ "error": { "code": "...", "message": "...", "details": [...] } }`. `details` is a list of fields only for validation errors.

2. Endpoint list

Method	URL	Login?	Purpose
GET	/api/health	No	Server and database status
POST	/api/auth/signup	No	Create an account
POST	/api/auth/login	No	Log in, get a token
POST	/api/auth/logout	Yes	Log out (cancel the token)
GET	/api/auth/me	Yes	Show the logged-in user
GET	/api/properties	No	List and filter properties
GET	/api/properties/recommended	Yes	Properties matching my saved preferences
GET	/api/properties/:propertyID	No	One property
POST	/api/properties	Yes	Add a property
PATCH	/api/properties/:propertyID	Owner	Change a property
DELETE	/api/properties/:propertyID	Owner	Delete a property and its images
POST	/api/properties/:propertyID/images	Owner	Upload images (multer)
DELETE	/api/properties/:propertyID/images	Owner	Remove one image
POST	/api/preferences	Yes	Save my preferences
GET	/api/preferences/me	Yes	See my preferences
PATCH	/api/preferences/me	Yes	Change my preferences

DELETE	/api/preferences/me	Yes	Delete my preferences
GET	/api/search	No / Yes	Search: filter, score and sort

3. Data models

Property (collection `properties`)

Field	Type	Rules
propertyID	String	Unique ID, made by the server (for example p_2353ddfe)
name	String	Required, 1 to 150 characters
location	Object	Required: address, city, locality (all text). Optional: pincode (6 digits), latitude (-90 to 90), longitude (-180 to 180)
price	Number	Required, 0 or more (rupees)
interiorFeatures	Object	Required: bhk (whole number 1 to 20), flooringType, furnishing (Unfurnished / Semi-furnished / Fully-furnished), areaSqFt (more than 0). Optional: style, modularKitchen (true/false), rooms (list of {type, lengthFt, widthFt})
images	List of text	Image links. Uploaded files look like /uploads/properties/<name>.png. Max 20.
availability	Boolean	true = available (default)
createdBy	String	userID of the owner. Set by the server.
createdAt	Date	Set by the server

User (collection `users`)

Field	Type	Rules
userID	String	Unique ID (for example u_2001)
username	String	3 to 30 letters, numbers or underscore. Unique.
email	String	Valid email, saved in lower case. Unique.
passwordHash	String	bcrypt hash. Never sent in any answer.
dateJoined	Date	Set by the server

User preference (collection `userPreferences`)

Field	Type	Rules
preferenceID	String	Made by the server (for example pr_3001)
userID	String	The owner. One record per user.
preferredLocation	List of text	Areas the user likes (required, can be empty)
budgetRange	Object	Required: { min, max }, numbers 0 or more, min not more than max
interiorPreferences	Object	Optional: flooringType (list), style (list), furnishing (list), minBhk (whole number), modularKitchen (true/false)

4. Health

GET `/api/health`

Shows if the server and database work.

Login needed: No

Success response: 200

```
{
  "status": "ok",
  "database": "disconnected",
  "uptimeSeconds": 1,
  "time": "2026-10-17T02:01:06.568Z"
}
```

5. Authentication

POST **`/api/auth/signup`**

Creates an account and logs the user in.

Login needed: No

Parameters

Name	Type	Meaning
username	Text	3 to 30 letters, numbers or underscore (_)
email	Text	A valid email
password	Text	8 to 72 characters, at least one letter and one number

Request body (JSON)

```
{
  "username": "new_user",
  "email": "new@example.com",
  "password": "Passw0rd123"
}
```

Example: bad values (400)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Some values are not valid",
    "details": [
      {
        "field": "username",
        "message": "must be 3 to 30 letters, numbers or underscore (_)"
      },
      {
        "field": "email",
        "message": "must be a valid email"
      },
      {
        "field": "password",
        "message": "must be 8 to 72 characters with at least one letter and one number"
      }
    ]
  }
}
```

Success response: 201

```
{
  "token": "eyJhbGciOiJIU...(JWT token)",
  "user": {
    "userID": "u_41dca1ca",
    "username": "new_user",
    "email": "new@example.com",
    "dateJoined": "2026-10-17T07:00:00.000Z"
  }
}
```

Errors

Status	Code	When
400	VALIDATION_ERROR	A field is wrong
409	CONFLICT	Email or username is already used
429	RATE_LIMITED	Too many tries

POST `/api/auth/login`

Logs in with email (or username) and password.

Login needed: No

Parameters

Name	Type	Meaning
email or username	Text	One of them
password	Text	Required

Request body (JSON)

```
{
  "email": "harsha@example.com",
  "password": "Test@1234"
}
```

Success response: 200

```
{
  "token": "eyJhbGciOiJIU...(JWT token)",
  "user": {
    "userID": "u_2001",
    "username": "harsha_10",
    "email": "harsha@example.com",
    "dateJoined": "2026-10-12T00:00:00.000Z"
  }
}
```

Errors

Status	Code	When
400	VALIDATION_ERROR	Missing field
401	INVALID_CREDENTIALS	Wrong email/username or password (same message for both, on purpose)
429	RATE_LIMITED	Too many tries

POST `/api/auth/logout`

Cancels the token. It cannot be used again.

Login needed: Yes

Success response: 200

```
{
  "message": "Logged out"
}
```

Errors

Status	Code	When
401	UNAUTHORIZED / INVALID_TOKEN / TOKEN_EXPIRED / TOKEN_REVOKED	Token problem

GET `/api/auth/me`

Shows the logged-in user. The password hash is never included.

Login needed: Yes

Example: after logout (401)

```
{
  "error": {
    "code": "TOKEN_REVOKED",
    "message": "You are logged out. Please log in again"
  }
}
```

Success response: 200

```
{
  "user": {
    "userID": "u_2001",
    "username": "harsha_10",
    "email": "harsha@example.com",
    "dateJoined": "2026-10-12T00:00:00.000Z"
  }
}
```

Errors

Status	Code	When
401	UNAUTHORIZED	No token
401	TOKEN_REVOKED	Logged out

6. Properties

GET `/api/properties`

Lists properties. All filters are optional and can be mixed. Cheapest first by default.

Login needed: No

Parameters

Name	Type	Meaning
minPrice, maxPrice	Number	Budget range (rupees)
bhk / minBhk	Whole number	Exact BHK, or this many or more
furnishing	Text list	Unfurnished, Semi-furnished, Fully-furnished. Comma list means any of them.
flooringType, style	Text list	Comma list means any of them. Not case sensitive.
modularKitchen, available	true / false	Filter on the feature or on availability
locality, city, q	Text	Area (comma list), city, or part of the name
sort	Text	price_asc (default), price_desc, newest
page, limit	Whole number	Page number (default 1) and page size (default 10, max 50)

Success response: 200


```
{
  "data": [
    {
      "propertyID": "p_1008",
      "name": "Skyline Towers 4BHK",
      "location": {
        "address": "Plot 9, Ring Road",
        "city": "Bengaluru",
        "locality": "Whitefield",
        "pincode": "560067",
        "latitude": 12.9855,
        "longitude": 77.731
      },
      "price": 21000000,
      "interiorFeatures": {
        "bhk": 4,
        "flooringType": "Marble",
        "furnishing": "Fully-furnished",
        "style": "Luxury",
        "modularKitchen": true,
        "areaSqFt": 2400,
        "rooms": [
          "..."
        ]
      },
      "images": [
        "https://example.com/images/p_1008_1.jpg",
        "https://example.com/images/p_1008_2.jpg"
      ],
      "availability": true,
      "createdAt": "2026-10-12",
      "createdBy": "u_2001"
    },
    {
      "propertyID": "p_1002",
      "name": "Green Valley 3BHK",
      "location": {
        "address": "45, 2nd Main",
        "city": "Bengaluru",
        "locality": "Koramangala",
        "pincode": "560034",
        "latitude": 12.9352,
        "longitude": 77.6245
      },
      "price": 13500000,
      "interiorFeatures": {
        "bhk": 3,
        "flooringType": "Wooden",
        "furnishing": "Fully-furnished",
        "style": "Luxury",
        "modularKitchen": true,
        "areaSqFt": 1650,
        "rooms": [
          "..."
        ]
      },
      "images": [
        "https://example.com/images/p_1002_1.jpg",
        "https://example.com/images/p_1002_2.jpg"
      ],
      "availability": true,
      "createdAt": "2026-10-12",
      "createdBy": "u_2001"
    }
  ],
  "total": 10,
  "page": 1,
  "limit": 2,
  "pages": 5
}
```


Errors

Status	Code	When

400	VALIDATION_ERROR	A filter value is wrong (for example limit=500)

Example: `/api/properties?minPrice=5000000&maxPrice=10000000&minBhk=2&furnishing=Semi-furnished,Fully-furnished&modularKitchen=true`

GET `/api/properties/recommended`

Uses the logged-in user's saved preferences (budget, locations, minimum BHK, furnishing, flooring, style, modular kitchen) as filters. Only available properties. Supports `sort`, `page`, `limit`. The answer also has `preferences`.

Login needed: Yes

Success response: 200


```
{
  "data": [
    {
      "propertyID": "p_1001",
      "name": "Sunrise Heights 2BHK",
      "location": {
        "address": "12, 5th Cross",
        "city": "Bengaluru",
        "locality": "HSR Layout",
        "pincode": "560102",
        "latitude": 12.9116,
        "longitude": 77.6474
      },
      "price": 8500000,
      "interiorFeatures": {
        "bhk": 2,
        "flooringType": "Vitrified",
        "furnishing": "Semi-furnished",
        "style": "Modern",
        "modularKitchen": true,
        "areaSqFt": 1100,
        "rooms": [
          "..."
        ]
      },
      "images": [
        "https://example.com/images/p_1001_1.jpg",
        "https://example.com/images/p_1001_2.jpg"
      ],
      "availability": true,
      "createdAt": "2026-10-12",
      "createdBy": "u_2001"
    }
  ],
  "total": 1,
  "page": 1,
  "limit": 2,
  "pages": 1,
  "preferences": {
    "preferredLocation": [
      "HSR Layout",
      "Indiranagar"
    ],
    "budgetRange": {
      "min": 4000000,
      "max": 9000000
    },
    "interiorPreferences": {
      "furnishing": [
        "Semi-furnished"
      ],
      "minBhk": 2,
      "modularKitchen": true
    },
    "preferenceID": "pr_ed9a8872",
    "userID": "u_2001"
  }
}
```

Errors

Status	Code	When
404	PREFERENCES_NOT_FOUND	The user has no saved preferences

GET `/api/properties/:propertyID`

Shows one property.

Login needed: No

Success response: 200


```
{
  "data": {
    "propertyID": "p_1001",
    "name": "Sunrise Heights 2BHK",
    "location": {
      "address": "12, 5th Cross",
      "city": "Bengaluru",
      "locality": "HSR Layout",
      "pincode": "560102",
      "latitude": 12.9116,
      "longitude": 77.6474
    },
    "price": 8500000,
    "interiorFeatures": {
      "bhk": 2,
      "flooringType": "Vitrified",
      "furnishing": "Semi-furnished",
      "style": "Modern",
      "modularKitchen": true,
      "areaSqFt": 1100,
      "rooms": [
        "..."
      ]
    },
    "images": [
      "https://example.com/images/p_1001_1.jpg",
      "https://example.com/images/p_1001_2.jpg"
    ],
    "availability": true,
    "createdAt": "2026-10-12",
    "createdBy": "u_2001"
  }
}
```


Errors

Status	Code	When
404	NOT_FOUND	No such property

POST `/api/properties`

Adds a property. The logged-in user becomes the owner. `propertyID`, `createdBy` and `createdAt` are set by the server (values sent for them are ignored). Images from a computer are added with the upload endpoint, not here.

Login needed: Yes

Request body (JSON)

```
{
  "name": "Test Flat",
  "price": 5000000,
  "location": {
    "address": "1 Main Rd",
    "city": "Bengaluru",
    "locality": "Indiranagar",
    "pincode": "560038"
  },
  "interiorFeatures": {
    "bhk": 2,
    "flooringType": "Vitrified",
    "furnishing": "Semi-furnished",
    "areaSqFt": 1000
  }
}
```

Example: bad values (400)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Some values are not valid",
    "details": [
      {
        "field": "location",
        "message": "is required"
      },
      {
        "field": "price",
        "message": "must be a number, 0 or more"
      },
      {
        "field": "interiorFeatures.bhk",
        "message": "must be a whole number from 1 to 20"
      },
      {
        "field": "interiorFeatures.flooringType",
        "message": "is required and must be text"
      },
      {
        "field": "interiorFeatures.furnishing",
        "message": "must be one of: Unfurnished, Semi-furnished, Fully-furnished"
      },
      {
        "field": "interiorFeatures.areaSqFt",
        "message": "must be a number more than 0"
      }
    ]
  }
}
```

Success response: 201


```
{
  "data": {
    "name": "Test Flat",
    "location": {
      "address": "1 Main Rd",
      "city": "Bengaluru",
      "locality": "Indiranagar",
      "pincode": "560038"
    },
    "price": 5000000,
    "interiorFeatures": {
      "bhk": 2,
      "flooringType": "Vitrified",
      "furnishing": "Semi-furnished",
      "areaSqFt": 1000
    },
    "propertyID": "p_08b67edd",
    "createdBy": "u_2001",
    "images": [],
    "availability": true,
    "createdAt": "2026-10-17T07:00:00.000Z"
  }
}
```


Errors

Status	Code	When
400	VALIDATION_ERROR	A field is wrong
401	UNAUTHORIZED	No token

PATCH `/api/properties/:propertyID`

Changes some fields. Send only what changes. A nested object (`location` or `interiorFeatures`) must be sent whole. Only the owner, or an admin (email in ADMIN_EMAILS), can do this.

Login needed: Owner

Request body (JSON)


```
{
  "price": 5200000,
  "availability": true
}
```


Example: another user (403)


```
{
  "error": {
    "code": "FORBIDDEN",
    "message": "Only the owner can change this property"
  }
}
```

Success response: 200

```
{
  "data": {
    "name": "Test Flat",
    "location": {
      "address": "1 Main Rd",
      "city": "Bengaluru",
      "locality": "Indiranagar",
      "pincode": "560038"
    },
    "price": 5200000,
    "interiorFeatures": {
      "bhk": 2,
      "flooringType": "Vitrified",
      "furnishing": "Semi-furnished",
      "areaSqFt": 1000
    },
    "propertyID": "p_08b67edd",
    "createdBy": "u_2001",
    "images": [],
    "availability": true,
    "createdAt": "2026-10-17T07:00:00.000Z"
  }
}
```

Errors

Status	Code	When
400	VALIDATION_ERROR	Wrong value, or an empty body
403	FORBIDDEN	Not the owner
404	NOT_FOUND	No such property

DELETE `/api/properties/:propertyID`

Deletes the property and its uploaded image files.

Login needed: Owner

Success response: 204

```
(no body)
```

Errors

Status	Code	When
403	FORBIDDEN	Not the owner
404	NOT_FOUND	No such property

7. Property images (multer)

POST `/api/properties/:propertyID/images`

Uploads 1 to 5 images as **multipart form data**, form field name `images`. Allowed: JPG, PNG, WEBP, up to 5 MB each. A property can have 20 images. The owner is checked **before** a file is saved. The server makes a random file name, and checks the first bytes of each file, so a fake image is rejected and deleted.

Login needed: Owner

Example (Windows)

```
curl.exe -X POST -H "Authorization: Bearer <token>" -F "images=@C:\\path\\room.jpg"
http://localhost:3000/api/properties/p_2353ddfe/images
```

Example: a fake image (400)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "A file is not a real image",
    "details": [
      {
        "field": "images",
        "message": "fake.png"
      }
    ]
  }
}
```

Example: bigger than 5 MB (413)

```
{
  "error": {
    "code": "FILE_TOO_LARGE",
    "message": "Each image must be 5 MB or smaller"
  }
}
```

Success response: 201

```
{
  "data": {
    "name": "Test Flat",
    "location": {
      "address": "1 Main Rd",
      "city": "Bengaluru",
      "locality": "Indiranagar",
      "pincode": "560038"
    },
    "price": 5200000,
    "interiorFeatures": {
      "bhk": 2,
      "flooringType": "Vitrified",
      "furnishing": "Semi-furnished",
      "areaSqFt": 1000
    },
    "propertyID": "p_08b67edd",
    "createdBy": "u_2001",
    "images": [
      "/uploads/properties/d73ebbe9b0dcba6ac3aa7b25.png"
    ],
    "availability": true,
    "createdAt": "2026-10-17T07:00:00.000Z"
  },
  "uploaded": [
    "/uploads/properties/d73ebbe9b0dcba6ac3aa7b25.png"
  ]
}
```

Errors

Status	Code	When
400	VALIDATION_ERROR	Not a real image, wrong type, no file, or more than 20 images on the property
400	UPLOAD_ERROR	More than 5 files, or wrong form field name
413	FILE_TOO_LARGE	A file is over 5 MB
403	FORBIDDEN	Not the owner
404	NOT_FOUND	No such property

DELETE `/api/properties/:propertyID/images`

Removes one image from the property and deletes its file.

Login needed: Owner

Request body (JSON)

```
{
  "image": "/uploads/properties/d73ebbe9b0dcba6ac3aa7b25.png"
}
```

Success response: 200

```
{
  "data": {
    "name": "Test Flat",
    "location": {
      "address": "1 Main Rd",
      "city": "Bengaluru",
      "locality": "Indiranagar",
      "pincode": "560038"
    },
    "price": 5200000,
    "interiorFeatures": {
      "bhk": 2,
      "flooringType": "Vitrified",
      "furnishing": "Semi-furnished",
      "areaSqFt": 1000
    },
    "propertyID": "p_08b67edd",
    "createdBy": "u_2001",
    "images": [],
    "availability": true,
    "createdAt": "2026-10-17T07:00:00.000Z"
  }
}
```

Errors

Status	Code	When
404	NOT_FOUND	The image is not on this property
403	FORBIDDEN	Not the owner

8. User preferences

Each user has one preference record. These endpoints always work on the logged-in user's own record.

POST `/api/preferences`

Saves the preferences. `userID` and `preferenceID` are set by the server.

Login needed: Yes

Request body (JSON)

```
{
  "preferredLocation": [
    "HSR Layout",
    "Indiranagar"
  ],
  "budgetRange": {
    "min": 5000000,
    "max": 10000000
  },
  "interiorPreferences": {
    "minBhk": 2,
    "furnishing": [
      "Semi-furnished"
    ],
    "modularKitchen": true
  }
}
```

Example: bad values (400)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Some values are not valid",
    "details": [
      {
        "field": "budgetRange",
        "message": "min must not be more than max"
      }
    ]
  }
}
```

Example: saving twice (409)

```
{
  "error": {
    "code": "CONFLICT",
    "message": "You already have preferences. Use PATCH /api/preferences/me to change them"
  }
}
```

Success response: 201

```
{
  "data": {
    "preferredLocation": [
      "HSR Layout",
      "Indiranagar"
    ],
    "budgetRange": {
      "min": 5000000,
      "max": 10000000
    },
    "interiorPreferences": {
      "furnishing": [
        "Semi-furnished"
      ],
      "minBhk": 2,
      "modularKitchen": true
    },
    "preferenceID": "pr_ed9a8872",
    "userID": "u_2001"
  }
}
```

Errors

Status	Code	When
400	VALIDATION_ERROR	A field is wrong (for example min more than max)
409	CONFLICT	Preferences exist already. Use PATCH.

GET `/api/preferences/me`

Shows the user's preferences.

Login needed: Yes

Example: nothing saved yet (404)

```
{
  "error": {
    "code": "PREFERENCES_NOT_FOUND",
    "message": "You have not saved preferences yet"
  }
}
```

Success response: 200

```
{
  "data": {
    "preferredLocation": [
      "HSR Layout",
      "Indiranagar"
    ],
    "budgetRange": {
      "min": 4000000,
      "max": 9000000
    },
    "interiorPreferences": {
      "furnishing": [
        "Semi-furnished"
      ],
      "minBhk": 2,
      "modularKitchen": true
    },
    "preferenceID": "pr_ed9a8872",
    "userID": "u_2001"
  }
}
```

Errors

Status	Code	When
404	PREFERENCES_NOT_FOUND	Nothing saved yet

PATCH `/api/preferences/me`

Changes some fields. A field that is sent replaces the old value of that field.

Login needed: Yes

Request body (JSON)

```
{
  "budgetRange": {
    "min": 4000000,
    "max": 9000000
  }
}
```

Success response: 200

```
{
  "data": {
    "preferredLocation": [
      "HSR Layout",
      "Indiranagar"
    ],
    "budgetRange": {
      "min": 4000000,
      "max": 9000000
    },
    "interiorPreferences": {
      "furnishing": [
        "Semi-furnished"
      ],
      "minBhk": 2,
      "modularKitchen": true
    },
    "preferenceID": "pr_ed9a8872",
    "userID": "u_2001"
  }
}
```

Errors

Status	Code	When
400	VALIDATION_ERROR	A field is wrong, or an empty body

404	PREFERENCES_NOT_FOUND	Nothing saved yet

DELETE `/api/preferences/me`

Deletes the user's preferences.

Login needed: Yes

Success response: 204

```
(no body)
```

Errors

Status	Code	When
404	PREFERENCES_NOT_FOUND	Nothing saved yet

9. Search

The search works in two steps. **Step 1** (MongoDB): budget, availability and city remove properties that cannot fit, and at most 500 of the cheapest candidates are kept. **Step 2**: every candidate gets a **relevance score from 0 to 100** from the wishes, and the list is sorted. Only the wishes that were given are counted. Without `strict=true`, a property that misses a wish is still shown, just lower. Each result lists which wishes `matched` and which were `missed`.

Wish	Points
Locality	25
BHK	20
Furnishing	15
Flooring	10
Style	10
Modular kitchen	10
Budget fit	10 (inside the budget: 4 to 10 points, cheaper is more)

Score = points earned / points possible x 100. With no wishes, `score` is `null` and the order is cheapest first. With the default sort, a tie goes to the cheaper property.

GET `/api/search`

Filters and ranks properties by budget and wishes.

Login needed: No (Yes when usePreferences=true)

Parameters

Name	Type	Meaning
usePreferences	true	Use my saved preferences as the wishes. Needs login. Values in the URL win.
strict	true	Every wish must match, otherwise the property is removed
minScore	0 to 100	Hide properties with a lower score
sort	Text	relevance (default), price_asc, price_desc, area_desc, location, newest
minPrice, maxPrice	Number	Budget range. Always removes properties outside it.

bhk, minBhk, furnishing, flooringType, style, modularKitchen, locality		The wishes (they give points)
city, available		Filters. available is true by default.
page, limit	Whole number	Paging, limit up to 50

The answer also has `sort`, `strict`, `criteria` (the wishes used) and `truncated` (true if more than 500 properties passed the budget filter).

Example: saved preferences, strict, cheapest first (/api/search?usePreferences=true&strict=true&sort=price_asc)


```
{
  "data": [
    {
      "propertyID": "p_1001",
      "name": "Sunrise Heights 2BHK",
      "location": {
        "address": "12, 5th Cross",
        "city": "Bengaluru",
        "locality": "HSR Layout",
        "pincode": "560102",
        "latitude": 12.9116,
        "longitude": 77.6474
      },
      "price": 8500000,
      "interiorFeatures": {
        "bhk": 2,
        "flooringType": "Vitrified",
        "furnishing": "Semi-furnished",
        "style": "Modern",
        "modularKitchen": true,
        "areaSqFt": 1100,
        "rooms": [
          "..."
        ]
      },
      "images": [
        "https://example.com/images/p_1001_1.jpg",
        "https://example.com/images/p_1001_2.jpg"
      ],
      "availability": true,
      "createdAt": "2026-10-12",
      "createdBy": "u_2001",
      "relevance": {
        "score": 93,
        "matched": [
          "locality",
          "bhk",
          "furnishing",
          "modularKitchen",
          "budget"
        ],
        "missed": []
      }
    }
  ],
  "total": 1,
  "page": 1,
  "limit": 10,
  "pages": 1,
  "sort": "price_asc",
  "strict": true,
  "criteria": {
    "budget": {
      "min": 4000000,
      "max": 9000000
    },
    "locality": [
      "HSR Layout",
      "Indiranagar"
    ],
    "minBhk": 2,
    "furnishing": [
      "Semi-furnished"
    ],
    "flooringType": [],
    "style": [],
    "modularKitchen": true
  },
  "truncated": false
}
```

Example: bad values (400)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Some search values are not valid",
    "details": [
      {
        "field": "minScore",
        "message": "must be from 0 to 100"
      },
      {
        "field": "sort",
        "message": "must be one of: relevance, price_asc, price_desc, area_desc, location, newest"
      }
    ]
  }
}
```

Success response: 200


```
{
  "data": [
    {
      "propertyID": "p_1004",
      "name": "Palm Residency 2BHK",
      "location": {
        "address": "27, Palm Street",
        "city": "Bengaluru",
        "locality": "Whitefield",
        "pincode": "560066",
        "latitude": 12.9698,
        "longitude": 77.75
      },
      "price": 7200000,
      "interiorFeatures": {
        "bhk": 2,
        "flooringType": "Vitrified",
        "furnishing": "Semi-furnished",
        "style": "Modern",
        "modularKitchen": true,
        "areaSqFt": 1050,
        "rooms": [
          "..."
        ]
      },
      "images": [
        "https://example.com/images/p_1004_1.jpg",
        "https://example.com/images/p_1004_2.jpg"
      ],
      "availability": true,
      "createdAt": "2026-10-12",
      "createdBy": "u_2001",
      "relevance": {
        "score": 95,
        "matched": [
          "bhk",
          "furnishing",
          "modularKitchen",
          "budget"
        ],
        "missed": []
      }
    },
    {
      "propertyID": "p_1001",
      "name": "Sunrise Heights 2BHK",
      "location": {
        "address": "12, 5th Cross",
        "city": "Bengaluru",
        "locality": "HSR Layout",
        "pincode": "560102",
        "latitude": 12.9116,
        "longitude": 77.6474
      },
      "price": 8500000,
      "interiorFeatures": {
        "bhk": 2,
        "flooringType": "Vitrified",
        "furnishing": "Semi-furnished",
        "style": "Modern",
        "modularKitchen": true,
        "areaSqFt": 1100,
        "rooms": [
          "..."
        ]
      },
      "images": [
        "https://example.com/images/p_1001_1.jpg",
        "https://example.com/images/p_1001_2.jpg"
      ],
      "availability": true,
      "createdAt": "2026-10-12",
      "createdBy": "u_2001",
      "relevance": {
```

```
        "score": 92,
        "matched": [
          "bhk",
          "furnishing",
          "modularKitchen",
          "budget"
        ],
        "missed": []
      }
    }
  ],
  "total": 6,
  "page": 1,
  "limit": 2,
  "pages": 3,
  "sort": "relevance",
  "strict": false,
  "criteria": {
    "budget": {
      "min": 5000000,
      "max": 10000000
    },
    "locality": [],
    "minBhk": 2,
    "furnishing": [
      "Semi-furnished"
    ],
    "flooringType": [],
    "style": [],
    "modularKitchen": true
  },
  "truncated": false
}
```


Errors

Status	Code	When
400	VALIDATION_ERROR	A value is wrong (sort, minScore, prices, ...)
401	UNAUTHORIZED	usePreferences=true without a token
404	PREFERENCES_NOT_FOUND	usePreferences=true and nothing saved

10. Error codes

Status	Code	When
400	VALIDATION_ERROR	A value is wrong. details lists each field.
400	INVALID_JSON	The body is not valid JSON
400	UPLOAD_ERROR	Wrong upload: too many files, or the wrong form field name
401	UNAUTHORIZED	No token, or not in the form Bearer <token>
401	INVALID_TOKEN	The token is fake or was changed
401	TOKEN_EXPIRED	The token is too old (7 days)
401	TOKEN_REVOKED	The user logged out with this token
401	INVALID_CREDENTIALS	Wrong email/username or password
403	FORBIDDEN	Only the owner (or an admin) can change this property
404	NOT_FOUND	Property or route not found
404	PREFERENCES_NOT_FOUND	The user has not saved preferences

409	CONFLICT	Email or username used already, or preferences exist already
413	FILE_TOO_LARGE	An image is bigger than 5 MB
429	RATE_LIMITED	More than 20 login or signup tries in 15 minutes
503	DB_UNAVAILABLE	MongoDB is not connected
500	SERVER_ERROR	Something went wrong on the server

Other examples:

Unknown URL (404)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Route not found: GET /api/nothing"
  }
}
```

Broken JSON body (400)

```
{
  "error": {
    "code": "INVALID_JSON",
    "message": "Request body is not valid JSON"
  }
}
```

11. Quick test in Windows PowerShell

```
$r = Invoke-RestMethod -Method Post -Uri http://localhost:3000/api/auth/login -ContentType "application/json" -Body
'{"email":"harsha@example.com","password":"Test@1234"}'
$h = @{ Authorization = "Bearer $($r.token)" }
Invoke-RestMethod "http://localhost:3000/api/search?minPrice=5000000&maxPrice=10000000&minBhk=2" | ConvertTo-Json -
Depth 6
Invoke-RestMethod "http://localhost:3000/api/search?usePreferences=true" -Headers $h | ConvertTo-Json -Depth 6
```

A Postman collection with all requests is in `docs/Property_API.postman_collection.json` in the source code. The examples in this document were produced by running the real API code.